Date:10/10/26

CO1-AT1 APPLICATION BASED QUESTIONS

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG.NO: 192324281 NAME: MONIKA R

Section 1: Regular Expressions – Resume Information Extraction

A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Algorithm

- Name: Extract the candidate's name. Example: Monika Ramesh.
- Email: Identify the email address. Example: monika@gmail.com.
- Mobile number: Extract the mobile number. Example: 9876543210.
- Technical skills: Detect skills such as Python, Java, SQL, Machine Learning, and NLP. Example: Python, Java.
- Experience: Extract the years of experience. Example: 3 years.
- Profile summary: Generate a structured summary of the candidate's details.
- Eligibility: Check whether the candidate has at least 2 years of experience and Python skill. Example: 3 years + Python = Eligible.

PYTHON CODE:

```
import re
resume = """
Name: Monika Ramesh
Email: monika@gmail.com
Phone: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""
name = re.search(r"Name:\s*(.*)", resume)
email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume)
phone = re.search(r"\b[6-9]\d{9}\b", resume)
skills = re.findall(
   r"Python|Java|SQL|Machine Learning|NLP", resume
)
experience = re.search(r"(\d+)\s*years?", resume)
print("Name:", name.group(1) if name else "Not found")
print("Email:", email.group() if email else "Not found")
print("Phone:", phone.group() if phone else "Not found")
print("Skills:", ", ".join(skills))
print("Experience:", experience.group() if experience else "Not found")
if experience and int(experience.group(1)) >= 2 and "Python" in skills:
   print("Eligible for shortlisting")
else: print("Not eligible for shortlisting")
```

OUTPUT:

Name: Monika Ramesh

Email: monika@gmail.com

Phone: 9876543210

Skills: Python, Java, SQL, Machine Learning, NLP

Experience: 3 years

Eligible for shortlisting

Section 2: Regular Expressions – Product Search System

An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Algorithm

- Exact keyword: Find products matching the keyword exactly. Example: Laptop → Laptop.
- Prefix search: Find products starting with the keyword. Example: Laptop → Laptop Bag.
- Suffix search: Find products ending with the keyword. Example: Laptop → Gaming Laptop.
- Partial search: Find products containing the keyword. Example: Laptop → Laptop Bag, Gaming Laptop.
- Case-insensitive search: Match keywords regardless of uppercase or lowercase. Example: laptop → Laptop.
- Matching products: Display all products matching the search.
- Search report: Display the total number of matching products. Example: Laptop → 3 products for partial search.

Python Code

```
import re
products = [
    "Laptop",
    "Laptop Bag",
    "Wireless Mouse",
    "Gaming Laptop",
    "Mouse Pad"
```

```
]
keyword = input("Enter search keyword: ")
exact = []
prefix = []
suffix = []
partial = []
for product in products:
    if re.fullmatch(re.escape(keyword), product, re.I):
        exact.append(product)
    if re.match(re.escape(keyword), product, re.I):
        prefix.append(product)
    if re.search(re.escape(keyword) + r"$", product, re.I):
        suffix.append(product)
    if re.search(re.escape(keyword), product, re.I):
        partial.append(product)
print("Exact matches:", exact)
print("Prefix matches:", prefix)
print("Suffix matches:", suffix)
print("Partial matches:", partial)
print("Total matching products:", len(partial))
```

Output:

Enter search keyword: **Laptop**

Exact matches: ['Laptop']

Prefix matches: ['Laptop', 'Laptop Bag']

Suffix matches: ['Laptop', 'Gaming Laptop']

Partial matches: ['Laptop', 'Laptop Bag', 'Gaming Laptop']

Total matching products: **3**

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Algorithm

- Register number: Check whether the student register number follows the required format. Example: 21CS123.
- Institutional email: Check whether the email follows the university's format. Example: monikar4281.sse@saveetha.com.
- Course code: Check whether the course code is valid. Example: DSA0308.
- Semester: Check whether the semester is within the permitted range. Example: Semester 3.
- Mobile number: Check whether the number contains 10 digits and starts with a valid digit.
- Validation messages: Display Valid or Invalid for each field.
- Final registration status: Registration is successful only if all required fields are valid.

Python Code:

```
import re
reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
if re.fullmatch(r"\d{2}[A-Z]{2}\d{3}", reg_no):
    print("Register Number: Valid")
```

```
    r1 = True
else:
    print("Register Number: Invalid")
    r1 = False
if re.fullmatch(r"[\w.-]+@saveetha\.com", email):
    print("Email: Valid")
    r2 = True
else:
    print("Email: Invalid")
    r2 = False
if re.fullmatch(r"[A-Z]{3}\d{4}", course):
    print("Course Code: Valid")
    r3 = True
else:
    print("Course Code: Invalid")
    r3 = False
if semester.isdigit() and 1 <= int(semester) <= 8:
    print("Semester: Valid")
    r4 = True
else:
    print("Semester: Invalid")
    r4 = False
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
    r5 = True
else:
    print("Mobile Number: Invalid")
    r5 = False
if r1 and r2 and r3 and r4 and r5:
    print("Registration Successful")
else:
    print("Registration Failed")
```

Output:

Enter Register Number: 19SD147

Enter Institutional Email: monikar4281.sse@saveetha.com

Enter Course Code: DSA0308

Enter Semester: 3

Enter Mobile Number: 9632587410

Register Number: Valid

Email: Valid

Course Code: Valid

Semester: Valid

Mobile Number: Valid

Registration Successful